

Sri Sivasubramaniya Nadar College of Engineering, Chennai

(An Autonomous Institution affiliated to Anna University)

Machine Learning Algorithms Laboratory

Experiment 5

Decision Tree and Random Forest: A Comparative Classification Study

Name	MAHALINGAM SS
Register No.	3122247001034
Degree & Branch	M.Tech (Integrated) Computer Science & Engineering
Semester	V
Subject Code & Name	ICS1512 – Machine Learning Algorithms Laboratory
Academic Year	2026–2027 (Odd)
Batch	2024–2029

Aim and Objective

To classify breast cancer cases as benign or malignant using Decision Tree and Random Forest classifiers and compare their performance using different evaluation metrics and cross-validation.

- To implement a Decision Tree classifier.
- To extend the Decision Tree into a Random Forest ensemble model.
- To study the impact of hyperparameters on overfitting and generalization.
- To select optimal hyperparameters using 5-Fold Cross-Validation.
- To compare single-tree and ensemble-tree models.

Dataset Description

The Wisconsin Diagnostic Breast Cancer dataset was used in this experiment. It contains 569 samples and 30 numerical attributes describing characteristics of breast cell nuclei. The target variable represents the diagnosis as benign or malignant.

- Total samples: 569

- Features used for classification: 30 numerical attributes
- Original columns: 32, including ID and diagnosis
- Target classes: Benign (B) and Malignant (M)
- Class distribution: 357 benign and 212 malignant

Theory

Decision Tree Classifier

A Decision Tree is a supervised learning model that recursively splits the feature space using impurity measures to form decision rules. Common splitting criteria include Gini Index and Entropy. Tree depth and node-splitting parameters control the complexity of the tree. Very deep trees can fit the training data too closely and may overfit.

Random Forest Classifier

Random Forest is an ensemble learning technique that combines multiple decision trees trained using bootstrapped samples and random feature selection. The individual tree predictions are combined using majority voting. Compared with a single Decision Tree, Random Forest generally reduces variance and improves generalization.

Preprocessing Steps

The following steps were performed in the notebook:

- Loaded the WDBC data from `wdbc.data`.
- Assigned names to the 30 feature columns together with ID and diagnosis.
- Checked dataset dimensions and information.
- Checked missing values and duplicate rows.
- Encoded diagnosis as 0 for benign and 1 for malignant.
- Removed the ID and diagnosis columns from the feature matrix.
- Performed exploratory data analysis using class distribution and correlation analysis.
- Split the data into training and testing sets using an 80:20 stratified split with `random_state=42`.

Exploratory Data Analysis

The dataset contains 569 rows and 32 columns before removing ID and diagnosis. No missing values and no duplicate rows were found.

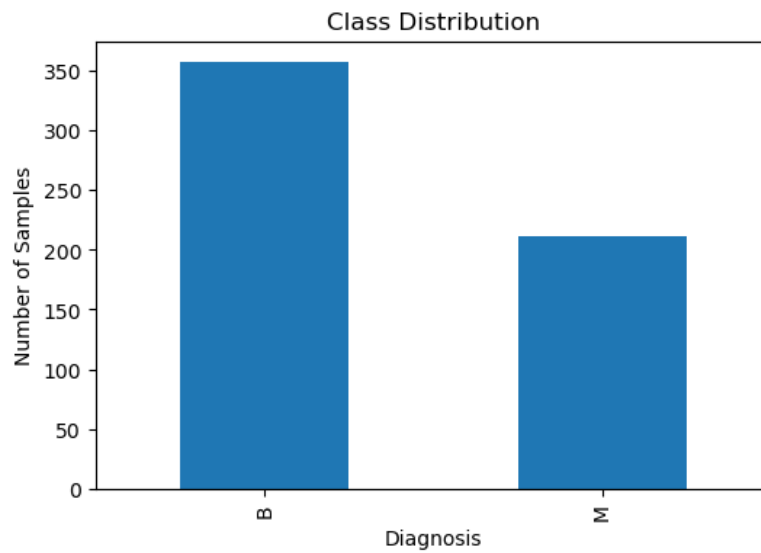


Figure 1: Class Distribution

The correlation matrix was also examined to understand the relationships among the numerical features.

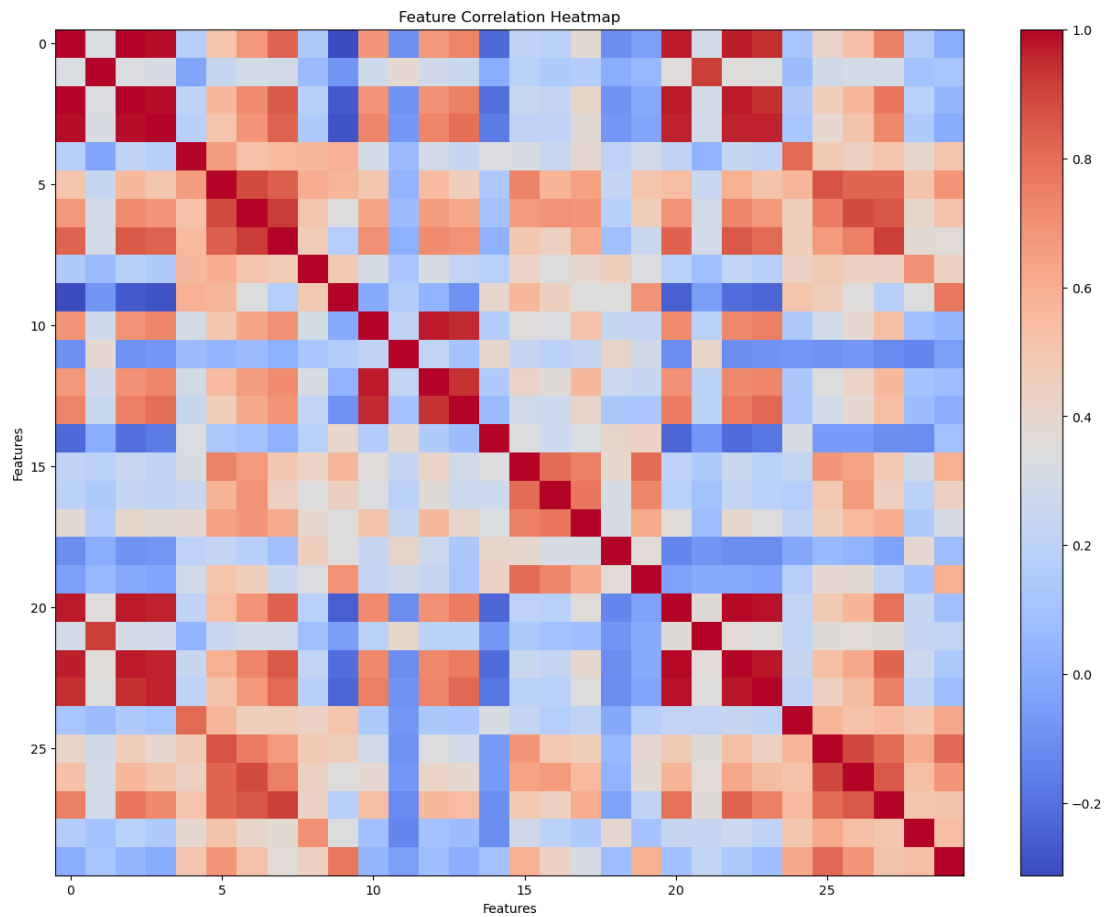


Figure 2: Feature Correlation Heatmap

Train-Test Split

An 80:20 stratified train-test split was used. This resulted in 455 training samples and 114 testing samples.

Decision Tree – Initial Model

A Decision Tree classifier was first trained using the default configuration with `random_state=42`.

Table 1: Initial Decision Tree Test Performance

Metric	Value
Accuracy	0.929825
Precision	0.904762
Recall	0.904762
F1 Score	0.904762

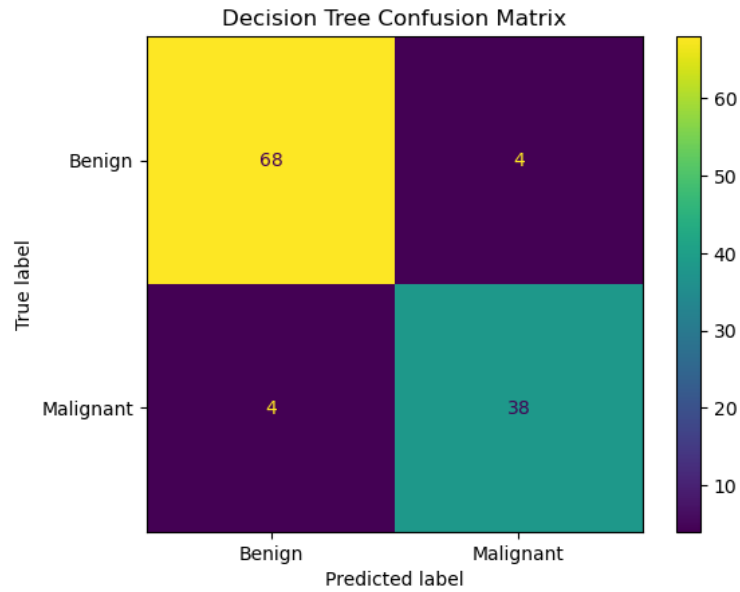


Figure 3: Initial Decision Tree Confusion Matrix

The complete initial Decision Tree structure generated by the notebook is shown below.

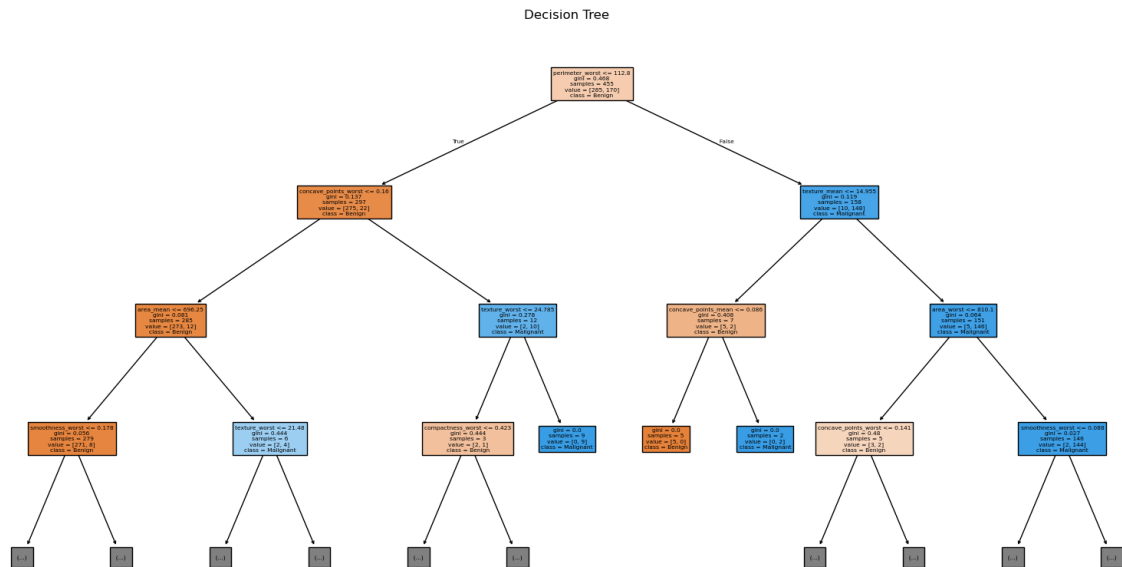


Figure 4: Initial Decision Tree Structure

Decision Tree Hyperparameter Tuning

A `StratifiedKfold` object with 5 folds, shuffling and `random_state=42` was used.

The Decision Tree search space included:

- Criterion: Gini and Entropy
- Maximum depth: 3, 5, 7, 10 and None
- Minimum samples split: 2, 5 and 10
- Minimum samples leaf: 1, 2 and 4

`GridSearchCV` was used with 5-fold cross-validation and accuracy as the scoring metric.

Table 2: Top Decision Tree Hyperparameter Results

Criterion	Max Depth	Min Split	Min Leaf	Avg CV Accuracy
Entropy	5	10	2	0.936264
Entropy	10	10	2	0.934066
Entropy	None	10	2	0.934066
Entropy	7	10	2	0.934066
Entropy	5	10	1	0.929670
Gini	10	5	2	0.929670
Gini	7	5	2	0.929670
Gini	None	5	2	0.929670
Entropy	None	2	2	0.929670
Entropy	None	5	2	0.929670

The best Decision Tree parameters obtained were:

```
criterion=entropy, max_depth=5, min_samples_split=10,
min_samples_leaf=2
```

The best average 5-fold CV accuracy was **0.936264 (93.63%)**.

The five Decision Tree F1 scores were:

```
0.865672, 0.941176, 0.875000, 0.939394, 0.939394
```

The average cross-validation F1 score was **0.912127**.

Optimized Decision Tree Results

Table 3: Optimized Decision Tree Test Performance

Metric	Value
Accuracy	0.964912
Precision	1.000000
Recall	0.904762
F1 Score	0.950000

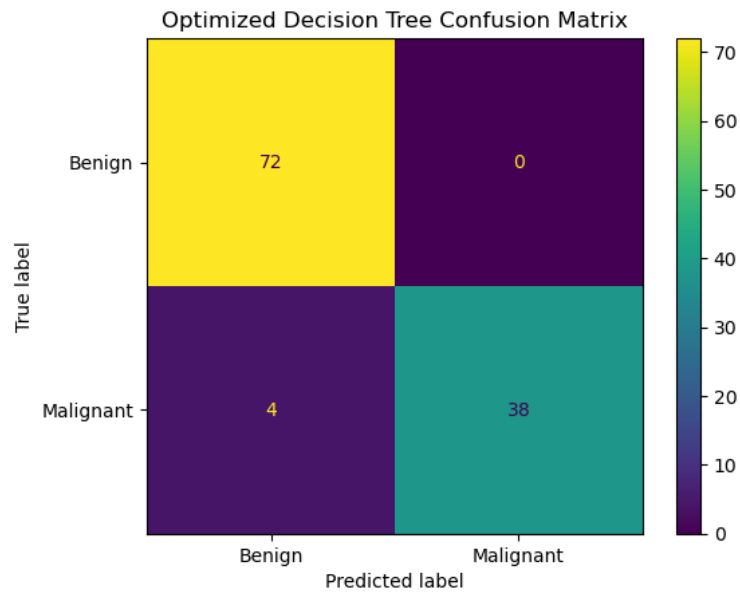


Figure 5: Optimized Decision Tree Confusion Matrix

Classification Report

	precision	recall	f1-score	support
Benign	0.95	1.00	0.97	72
Malignant	1.00	0.90	0.95	42
accuracy			0.96	114
macro avg	0.97	0.95	0.96	114
weighted avg	0.97	0.96	0.96	114

Random Forest – Initial Model

A Random Forest classifier was trained using `random_state=42`.

Table 4: Initial Random Forest Test Performance

Metric	Value
Accuracy	0.973684
Precision	1.000000
Recall	0.928571
F1 Score	0.962963

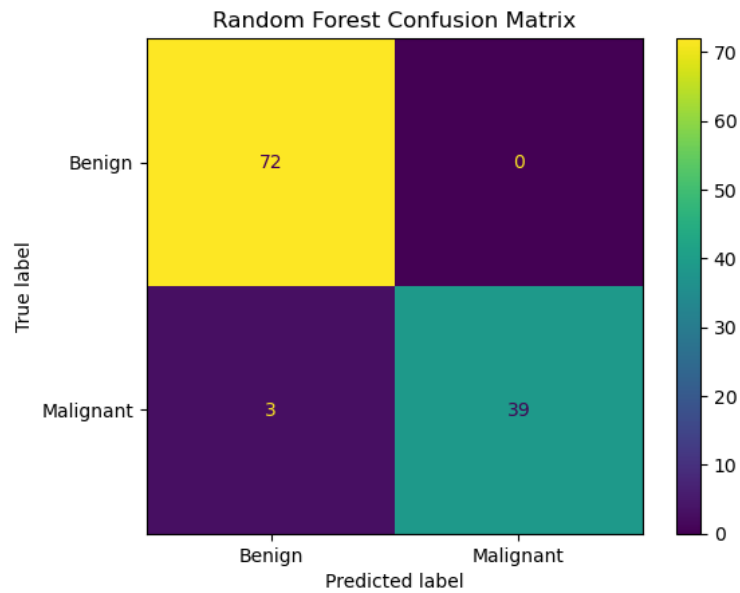


Figure 6: Initial Random Forest Confusion Matrix

Random Forest Hyperparameter Tuning

The Random Forest search space used in the final tuning section of the notebook was:

- Number of estimators: 50, 100
- Maximum depth: 5, 10 and None
- Maximum features: sqrt and log2
- Bootstrap: True

The search used `GridSearchCV` with 5-fold stratified cross-validation.

Table 5: Top Random Forest Hyperparameter Results

N Estimators	Max Depth	Max Features	Bootstrap	Avg CV Accuracy
100	5	sqrt	True	0.967033
100	10	sqrt	True	0.964835
100	None	sqrt	True	0.964835
100	10	log2	True	0.964835
100	None	log2	True	0.964835
50	5	sqrt	True	0.962637
50	10	sqrt	True	0.960440
50	None	sqrt	True	0.960440
100	5	log2	True	0.960440
50	10	log2	True	0.960440

The best Random Forest parameters obtained were:

`n_estimators=100, max_depth=5, max_features=sqrt, bootstrap=True`

The best average 5-fold CV accuracy was **0.967033 (96.70%)**.

The five Random Forest F1 scores were:

0.955224, 1.000000, 0.923077, 0.927536, 0.970588

The average cross-validation F1 score was **0.955285**.

Optimized Random Forest Results

Table 6: Optimized Random Forest Test Performance

Metric	Value
Accuracy	0.973684
Precision	1.000000
Recall	0.928571
F1 Score	0.962963

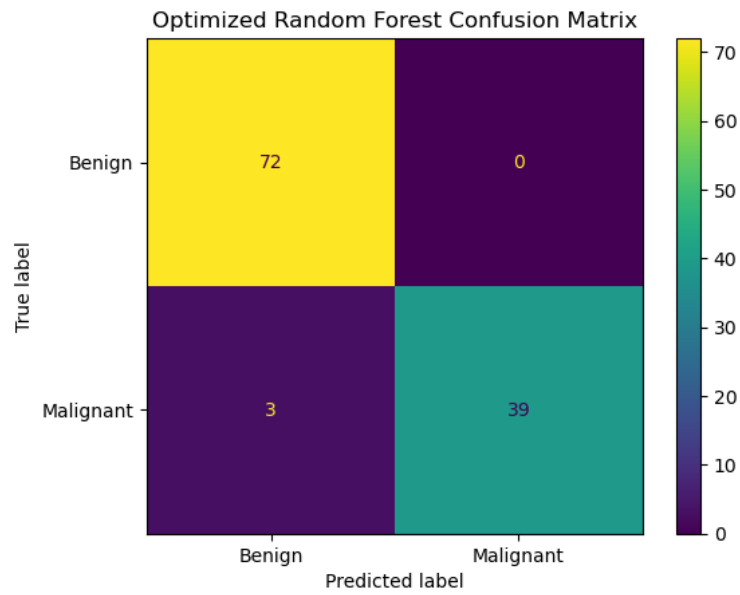


Figure 7: Optimized Random Forest Confusion Matrix

Classification Report

	precision	recall	f1-score	support
Benign	0.96	1.00	0.98	72
Malignant	1.00	0.93	0.96	42
accuracy			0.97	114
macro avg	0.98	0.96	0.97	114
weighted avg	0.97	0.97	0.97	114

5-Fold Cross-Validation Performance Comparison

Fold	DT Accuracy	DT F1	RF Accuracy	RF F1
1	0.901099	0.865672	0.967033	0.955224
2	0.956044	0.941176	1.000000	1.000000
3	0.912088	0.875000	0.945055	0.923077
4	0.956044	0.939394	0.945055	0.927536
5	0.956044	0.939394	0.978022	0.970588

Final Performance Comparison

Table 8: Decision Tree vs Random Forest

Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
Decision Tree	0.964912	1.000000	0.904762	0.950000	0.954696
Random Forest	0.973684	1.000000	0.928571	0.962963	0.992063

The optimized Random Forest achieved higher test accuracy, recall, F1 score and ROC-AUC than the optimized Decision Tree. Both models achieved a precision of 1.0000 on the test set.

ROC Curve and AUC

The notebook computed the ROC curve points and ROC-AUC values for both optimized models. The resulting AUC values were:

- Decision Tree: 0.954696
- Random Forest: 0.992063

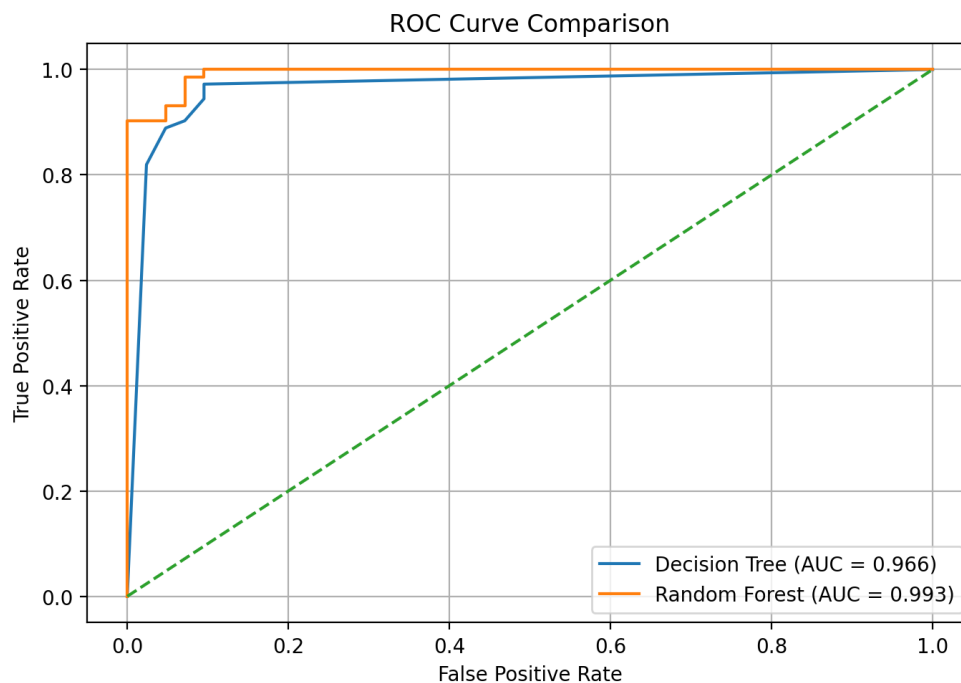


Figure 8: ROC Curve Comparison of Decision Tree and Random Forest

The ROC-AUC result indicates better class-separation performance for the Random Forest model.

Observation Questions

1. How does tree depth affect overfitting in Decision Trees?

Increasing the tree depth allows the model to create more complex decision rules. A very deep tree can fit the training data closely and may overfit, while restricting the depth can improve generalization.

2. Which hyperparameter had the greatest impact on performance?

From the obtained cross-validation results, the combination of criterion, maximum depth, minimum samples split and minimum samples leaf affected Decision Tree performance. For the best Decision Tree, entropy with maximum depth 5, minimum samples split 10 and minimum samples leaf 2 produced the highest average CV accuracy. For Random Forest, the best combination used 100 estimators, maximum depth 5 and square-root feature selection.

3. How does Random Forest improve generalization?

Random Forest trains multiple decision trees using bootstrap samples and random subsets of features. Aggregating their predictions reduces the variance of an individual tree and generally improves stability and generalization.

4. Did ensemble learning always improve performance? Why or why not?

For the obtained test results, Random Forest performed better than the optimized Decision Tree. However, ensemble learning does not guarantee improvement in every dataset or configuration because its performance depends on the base learners, hyperparameters, data characteristics and amount of variance present.

Observations

- The initial Decision Tree achieved a test accuracy of 92.98%.
- Hyperparameter tuning improved the Decision Tree test accuracy to 96.49%.
- The optimized Decision Tree obtained a CV accuracy of 93.63% and CV F1 score of 0.9121.
- Random Forest achieved a test accuracy of 97.37%.
- The optimized Random Forest obtained a CV accuracy of 96.70% and CV F1 score of 0.9553.
- Random Forest achieved a higher ROC-AUC of 0.9921 compared with 0.9547 for the Decision Tree.

- The results show the benefit of combining multiple trees to reduce variance and improve generalization.

Learning Outcomes

- Understood the working of Decision Tree classification.
- Learned how Random Forest combines multiple decision trees.
- Performed hyperparameter tuning using GridSearchCV.
- Understood the use of Stratified K-Fold cross-validation.
- Evaluated classification models using accuracy, precision, recall, F1 score and ROC-AUC.
- Compared the performance of Decision Tree and Random Forest.

Conclusion

Decision Tree and Random Forest classifiers were implemented using the Wisconsin Diagnostic Breast Cancer dataset. Hyperparameter tuning was performed using GridSearchCV and five-fold stratified cross-validation. The optimized Decision Tree achieved 96.49% test accuracy, while the optimized Random Forest achieved 97.37% test accuracy. Random Forest also produced higher recall, F1 score and ROC-AUC. The results demonstrate that the ensemble approach provided better generalization and more stable classification performance than the single Decision Tree for this dataset.

References

- Scikit-learn: Decision Trees
- Scikit-learn: Random Forest
- UCI Machine Learning Repository: Wisconsin Diagnostic Breast Cancer Dataset

Complete Python Code

The following section contains all the code cells from the uploaded Assignment 5 notebook in their original order. No notebook code cells have been intentionally omitted.

```
% Code Cell 1
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import (
    train_test_split,
    StratifiedKFold,
    GridSearchCV,
    cross_val_score
)

from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    ConfusionMatrixDisplay,
    classification_report,
    roc_curve,
    roc_auc_score
)

print("Libraries imported successfully!")

% Code Cell 2
data = pd.read_csv(
    "wdbc.data",
    header=None
)

print("Dataset loaded successfully!")
print("Shape:", data.shape)

% Code Cell 3
feature_names = [
    "radius_mean",
```

```

        "texture_mean",
        "perimeter_mean",
        "area_mean",
        "smoothness_mean",
        "compactness_mean",
        "concavity_mean",
        "concave_points_mean",
        "symmetry_mean",
        "fractal_dimension_mean",

        "radius_se",
        "texture_se",
        "perimeter_se",
        "area_se",
        "smoothness_se",
        "compactness_se",
        "concavity_se",
        "concave_points_se",
        "symmetry_se",
        "fractal_dimension_se",

        "radius_worst",
        "texture_worst",
        "perimeter_worst",
        "area_worst",
        "smoothness_worst",
        "compactness_worst",
        "concavity_worst",
        "concave_points_worst",
        "symmetry_worst",
        "fractal_dimension_worst"
    ]

columns = ["id", "diagnosis"] + feature_names

data.columns = columns

print(data.head())

% Code Cell 4
print("Number of rows:", data.shape[0])
print("Number of columns:", data.shape[1])

print("\nDataset information:")
data.info()

% Code Cell 5

```



```

data.head()

% Code Cell 6
print("Total missing values:", data.isnull().sum().sum())

% Code Cell 7
data.isnull().sum()

% Code Cell 8
print("Number of duplicate rows:", data.duplicated().sum())

% Code Cell 9
print(data["diagnosis"].value_counts())

% Code Cell 10
data["diagnosis"].value_counts().plot(
    kind="bar",
    figsize=(6, 4)
)

plt.title("Class Distribution")
plt.xlabel("Diagnosis")
plt.ylabel("Number of Samples")
plt.show()

% Code Cell 11
data["diagnosis"] = data["diagnosis"].map({
    "B": 0,
    "M": 1
})

print(data["diagnosis"].value_counts())

% Code Cell 12
X = data.drop(
    columns=["id", "diagnosis"]
)

y = data["diagnosis"]

print("X shape:", X.shape)
print("y shape:", y.shape)

% Code Cell 13
correlation_matrix = X.corr()

plt.figure(figsize=(16, 12))

```

```

plt.imshow(
    correlation_matrix,
    cmap="coolwarm",
    aspect="auto"
)

plt.colorbar()

plt.title("Feature Correlation Heatmap")
plt.xlabel("Features")
plt.ylabel("Features")

plt.show()

% Code Cell 14
X_train, X_test, y_train, y_test = train_test_split(
    X,
    Y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training set:", X_train.shape)
print("Testing set :", X_test.shape)

% Code Cell 15
dt_model = DecisionTreeClassifier(
    random_state=42
)

dt_model.fit(
    X_train,
    y_train
)

print("Decision Tree trained successfully!")

% Code Cell 16
y_pred_dt = dt_model.predict(X_test)

print("Predictions:")
print(y_pred_dt)

% Code Cell 17
dt_accuracy = accuracy_score(

```

```

        y_test,
        y_pred_dt
    )

    dt_precision = precision_score(
        y_test,
        y_pred_dt
    )

    dt_recall = recall_score(
        y_test,
        y_pred_dt
    )

    dt_f1 = f1_score(
        y_test,
        y_pred_dt
    )

    print("Decision Tree Results")
    print("-----")
    print("Accuracy :", dt_accuracy)
    print("Precision:", dt_precision)
    print("Recall   :", dt_recall)
    print("F1 Score :", dt_f1)

    % Code Cell 18
    print(
        classification_report(
            y_test,
            y_pred_dt,
            target_names=["Benign", "Malignant"]
        )
    )

    % Code Cell 19
    cm_dt = confusion_matrix(
        y_test,
        y_pred_dt
    )

    disp = ConfusionMatrixDisplay(
        confusion_matrix=cm_dt,
        display_labels=["Benign", "Malignant"]
    )

    disp.plot()

```

```

plt.title("Decision Tree Confusion Matrix")

plt.show()

% Code Cell 20
plt.figure(figsize=(20, 10))

plot_tree(
    dt_model,
    feature_names=X.columns,
    class_names=["Benign", "Malignant"],
    filled=True,
    max_depth=3
)

plt.title("Decision Tree")

plt.show()

% Code Cell 21
cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

print("5-Fold Cross-Validation created successfully!")

% Code Cell 22
dt_param_grid = {
    "criterion": ["gini", "entropy"],
    "max_depth": [3, 5, 7, 10, None],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf": [1, 2, 4]
}

print("Decision Tree parameter grid:")
print(dt_param_grid)

% Code Cell 23
dt_grid = GridSearchCV(
    estimator=DecisionTreeClassifier(random_state=42),
    param_grid=dt_param_grid,
    cv=cv,
    scoring="accuracy",
    n_jobs=-1

```

```

)

dt_grid.fit(X_train, y_train)

print("Decision Tree hyperparameter tuning completed!")

% Code Cell 24
print("Best Decision Tree Parameters:")
print(dt_grid.best_params_)

print("\nBest Average CV Accuracy:")
print(dt_grid.best_score_)

% Code Cell 25
best_dt = dt_grid.best_estimator_

print("Best Decision Tree:")
print(best_dt)

% Code Cell 26
dt_results = pd.DataFrame(
    dt_grid.cv_results_
)

dt_results = dt_results[
    [
        "param_criterion",
        "param_max_depth",
        "param_min_samples_split",
        "param_min_samples_leaf",
        "mean_test_score"
    ]
]

dt_results = dt_results.sort_values(
    by="mean_test_score",
    ascending=False
)

dt_results.head(10)

% Code Cell 27
dt_results.columns = [
    "Criterion",
    "Max Depth",
    "Min Samples Split",
    "Min Samples Leaf",

```

```

        "Avg CV Accuracy"
    ]

dt_results.head(10)

% Code Cell 28
dt_f1_scores = cross_val_score(
    best_dt,
    X_train,
    y_train,
    cv=cv,
    scoring="f1"
)

print("Decision Tree F1 Scores for 5 Folds:")
print(dt_f1_scores)

print("\nAverage CV F1 Score:")
print(dt_f1_scores.mean())

% Code Cell 29
best_dt.fit(
    X_train,
    y_train
)

print("Optimized Decision Tree trained successfully!")

% Code Cell 30
y_pred_best_dt = best_dt.predict(
    X_test
)

print("Optimized Decision Tree predictions generated!")

% Code Cell 31
best_dt_accuracy = accuracy_score(
    y_test,
    y_pred_best_dt
)

best_dt_precision = precision_score(
    y_test,
    y_pred_best_dt
)

best_dt_recall = recall_score(

```

```

        y_test,
        y_pred_best_dt
    )

best_dt_f1 = f1_score(
    y_test,
    y_pred_best_dt
)

print("Optimized Decision Tree")
print("-----")
print("Accuracy :", best_dt_accuracy)
print("Precision:", best_dt_precision)
print("Recall   :", best_dt_recall)
print("F1 Score :", best_dt_f1)

% Code Cell 32
cm_best_dt = confusion_matrix(
    y_test,
    y_pred_best_dt
)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm_best_dt,
    display_labels=["Benign", "Malignant"]
)

disp.plot()

plt.title("Optimized Decision Tree Confusion Matrix")

plt.show()

% Code Cell 33
print(
    classification_report(
        y_test,
        y_pred_best_dt,
        target_names=["Benign", "Malignant"]
    )
)

% Code Cell 34
rf_model = RandomForestClassifier(
    random_state=42
)

```

```

rf_model.fit(
    X_train,
    y_train
)

print("Random Forest trained successfully!")

% Code Cell 35
y_pred_rf = rf_model.predict(X_test)

print("Random Forest predictions generated!")

% Code Cell 36
rf_accuracy = accuracy_score(
    y_test,
    y_pred_rf
)

rf_precision = precision_score(
    y_test,
    y_pred_rf
)

rf_recall = recall_score(
    y_test,
    y_pred_rf
)

rf_f1 = f1_score(
    y_test,
    y_pred_rf
)

print("Random Forest Results")
print("-----")
print("Accuracy :", rf_accuracy)
print("Precision:", rf_precision)
print("Recall   :", rf_recall)
print("F1 Score :", rf_f1)

% Code Cell 37
print(
    classification_report(
        y_test,
        y_pred_rf,
        target_names=["Benign", "Malignant"]
    )
)

```



```

)

% Code Cell 38
cm_rf = confusion_matrix(
    y_test,
    y_pred_rf
)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm_rf,
    display_labels=["Benign", "Malignant"]
)

disp.plot()

plt.title("Random Forest Confusion Matrix")

plt.show()

% Code Cell 39
rf_param_grid = {
    "n_estimators": [50, 100, 200],
    "max_depth": [None, 5, 10],
    "max_features": ["sqrt", "log2"],
    "bootstrap": [True, False]
}

print("Random Forest parameter grid:")
print(rf_param_grid)

% Code Cell 40
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV

print("Random Forest libraries imported!")

% Code Cell 41
from sklearn.model_selection import StratifiedKFold

cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

print("5-Fold Cross-Validation created!")

```

```

% Code Cell 42
rf_param_grid = {
    "n_estimators": [50, 100, 200],
    "max_depth": [None, 5, 10],
    "max_features": ["sqrt", "log2"],
    "bootstrap": [True, False]
}

print("Random Forest parameter grid created!")

% Code Cell 43
import pandas as pd

data = pd.read_csv(
    "wdbc.data",
    header=None
)

print("Dataset loaded!")
print("Shape:", data.shape)

% Code Cell 44
feature_names = [
    "radius_mean",
    "texture_mean",
    "perimeter_mean",
    "area_mean",
    "smoothness_mean",
    "compactness_mean",
    "concavity_mean",
    "concave_points_mean",
    "symmetry_mean",
    "fractal_dimension_mean",

    "radius_se",
    "texture_se",
    "perimeter_se",
    "area_se",
    "smoothness_se",
    "compactness_se",
    "concavity_se",
    "concave_points_se",
    "symmetry_se",
    "fractal_dimension_se",

    "radius_worst",
    "texture_worst",

```

```

        "perimeter_worst",
        "area_worst",
        "smoothness_worst",
        "compactness_worst",
        "concavity_worst",
        "concave_points_worst",
        "symmetry_worst",
        "fractal_dimension_worst"
    ]

data.columns = ["id", "diagnosis"] + feature_names

print(data.shape)
print(data.head())

% Code Cell 45
data["diagnosis"] = data["diagnosis"].map({
    "B": 0,
    "M": 1
})

print(data["diagnosis"].value_counts())

% Code Cell 46
X = data.drop(
    columns=["id", "diagnosis"]
)

y = data["diagnosis"]

print("X:", X.shape)
print("y:", y.shape)

% Code Cell 47
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training:", X_train.shape)
print("Testing :", X_test.shape)

```

```

% Code Cell 48
from sklearn.model_selection import StratifiedKFold

cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

print("5-Fold CV ready!")

% Code Cell 49
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier

rf_param_grid = {
    "n_estimators": [50, 100, 200],
    "max_depth": [None, 5, 10],
    "max_features": ["sqrt", "log2"],
    "bootstrap": [True, False]
}

print("Random Forest settings ready!")

% Code Cell 50
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier

print("Imports successful!")

% Code Cell 51
from sklearn.model_selection import StratifiedKFold

cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

print("cv created successfully!")

% Code Cell 52
print(cv)

% Code Cell 53
import pandas as pd

```

```

data = pd.read_csv(
    "wdbc.data",
    header=None
)

print("Dataset loaded!")
print("Shape:", data.shape)

% Code Cell 54
feature_names = [
    "radius_mean",
    "texture_mean",
    "perimeter_mean",
    "area_mean",
    "smoothness_mean",
    "compactness_mean",
    "concavity_mean",
    "concave_points_mean",
    "symmetry_mean",
    "fractal_dimension_mean",
    "radius_se",
    "texture_se",
    "perimeter_se",
    "area_se",
    "smoothness_se",
    "compactness_se",
    "concavity_se",
    "concave_points_se",
    "symmetry_se",
    "fractal_dimension_se",
    "radius_worst",
    "texture_worst",
    "perimeter_worst",
    "area_worst",
    "smoothness_worst",
    "compactness_worst",
    "concavity_worst",
    "concave_points_worst",
    "symmetry_worst",
    "fractal_dimension_worst"
]

data.columns = ["id", "diagnosis"] + feature_names

print(data.head())

% Code Cell 55

```

```

data["diagnosis"] = data["diagnosis"].map({
    "B": 0,
    "M": 1
})

print(data["diagnosis"].value_counts())

% Code Cell 56
X = data.drop(columns=["id", "diagnosis"])
y = data["diagnosis"]

print("X shape:", X.shape)
print("y shape:", y.shape)

% Code Cell 57
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("X_train:", X_train.shape)
print("X_test :", X_test.shape)

% Code Cell 58
from sklearn.model_selection import StratifiedKFold

cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

print("5-Fold CV created!")

% Code Cell 59
rf_param_grid = {
    "n_estimators": [50, 100],
    "max_depth": [5, 10, None],
    "max_features": ["sqrt", "log2"],
    "bootstrap": [True]
}

```

```

print("Parameter grid ready!")

% Code Cell 60
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier

print("Imports successful!")

% Code Cell 61
from sklearn.model_selection import StratifiedKFold

cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

% Code Cell 62
import pandas as pd

data = pd.read_csv(
    "wdbc.data",
    header=None
)

print("Dataset loaded successfully!")
print("Shape:", data.shape)

% Code Cell 63
# Add column names

feature_names = [
    "radius_mean", "texture_mean", "perimeter_mean", "area_mean",
    "smoothness_mean", "compactness_mean", "concavity_mean",
    "concave_points_mean", "symmetry_mean", "fractal_dimension_mean",

    "radius_se", "texture_se", "perimeter_se", "area_se",
    "smoothness_se", "compactness_se", "concavity_se",
    "concave_points_se", "symmetry_se", "fractal_dimension_se",

    "radius_worst", "texture_worst", "perimeter_worst", "area_worst",
    "smoothness_worst", "compactness_worst", "concavity_worst",
    "concave_points_worst", "symmetry_worst", "fractal_dimension_worst"
]

data.columns = ["id", "diagnosis"] + feature_names

```

```

# Encode target
data["diagnosis"] = data["diagnosis"].map({
    "B": 0,
    "M": 1
})

# Create X and y
X = data.drop(columns=["id", "diagnosis"])
y = data["diagnosis"]

# Train-test split
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("X:", X.shape)
print("y:", y.shape)
print("X_train:", X_train.shape)
print("X_test:", X_test.shape)

% Code Cell 64
from sklearn.model_selection import StratifiedKFold, GridSearchCV
from sklearn.ensemble import RandomForestClassifier

cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

rf_param_grid = {
    "n_estimators": [50, 100],
    "max_depth": [5, 10, None],
    "max_features": ["sqrt", "log2"],
    "bootstrap": [True]
}

rf_grid = GridSearchCV(
    estimator=RandomForestClassifier(random_state=42),
    param_grid=rf_param_grid,
    cv=cv,

```



```

        scoring="accuracy",
        n_jobs=-1
    )

    rf_grid.fit(X_train, y_train)

    print("Random Forest hyperparameter tuning completed!")

    % Code Cell 65
    print("Best Random Forest Parameters:")
    print(rf_grid.best_params_)

    print("\nBest Average CV Accuracy:")
    print(rf_grid.best_score_)

    % Code Cell 66
    best_rf = rf_grid.best_estimator_

    print("Best Random Forest:")
    print(best_rf)

    % Code Cell 67
    rf_results = pd.DataFrame(
        rf_grid.cv_results_
    )

    rf_results = rf_results[
        [
            "param_n_estimators",
            "param_max_depth",
            "param_max_features",
            "param_bootstrap",
            "mean_test_score"
        ]
    ]

    rf_results = rf_results.sort_values(
        by="mean_test_score",
        ascending=False
    )

    rf_results.head(10)

    % Code Cell 68
    rf_results.columns = [
        "N Estimators",
        "Max Depth",

```

```

        "Max Features",
        "Bootstrap",
        "Avg CV Accuracy"
    ]

rf_results.head(10)

% Code Cell 69
from sklearn.model_selection import cross_val_score

print("cross_val_score imported!")

% Code Cell 70
rf_f1_scores = cross_val_score(
    best_rf,
    X_train,
    y_train,
    cv=cv,
    scoring="f1"
)

print("Random Forest F1 Scores:")
print(rf_f1_scores)

print("\nAverage CV F1 Score:")
print(rf_f1_scores.mean())

% Code Cell 71
best_rf.fit(
    X_train,
    y_train
)

print("Optimized Random Forest trained successfully!")

% Code Cell 72
y_pred_best_rf = best_rf.predict(
    X_test
)

print("Optimized Random Forest predictions generated!")

% Code Cell 73
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,

```

```

        f1_score
    )

print("Metrics imported successfully!")

% Code Cell 74
best_rf_accuracy = accuracy_score(
    y_test,
    y_pred_best_rf
)

best_rf_precision = precision_score(
    y_test,
    y_pred_best_rf
)

best_rf_recall = recall_score(
    y_test,
    y_pred_best_rf
)

best_rf_f1 = f1_score(
    y_test,
    y_pred_best_rf
)

print("Optimized Random Forest")
print("-----")
print("Accuracy :", best_rf_accuracy)
print("Precision:", best_rf_precision)
print("Recall    :", best_rf_recall)
print("F1 Score :", best_rf_f1)

% Code Cell 75
# All required imports for the remaining assignment

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import (
    train_test_split,
    StratifiedKFold,
    GridSearchCV,
    cross_val_score
)

```

```

from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    ConfusionMatrixDisplay,
    classification_report,
    roc_curve,
    roc_auc_score
)

print("All required libraries imported successfully!")

% Code Cell 76
cm_best_rf = confusion_matrix(
    y_test,
    y_pred_best_rf
)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm_best_rf,
    display_labels=["Benign", "Malignant"]
)

disp.plot()

plt.title("Optimized Random Forest Confusion Matrix")

plt.show()

% Code Cell 77
print(
    classification_report(
        y_test,
        y_pred_best_rf,
        target_names=["Benign", "Malignant"]
    )
)

% Code Cell 78
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import StratifiedKFold, GridSearchCV

```

```

print("Decision Tree tools imported!")

% Code Cell 79
dt_param_grid = {
    "criterion": ["gini", "entropy"],
    "max_depth": [3, 5, 7, 10, None],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf": [1, 2, 4]
}

print("Decision Tree parameter grid created!")

% Code Cell 80
cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

% Code Cell 81
dt_grid = GridSearchCV(
    estimator=DecisionTreeClassifier(random_state=42),
    param_grid=dt_param_grid,
    cv=cv,
    scoring="accuracy",
    n_jobs=-1
)

dt_grid.fit(X_train, y_train)

print("Decision Tree tuning completed!")

% Code Cell 82
best_dt = dt_grid.best_estimator_

print("Best Decision Tree Parameters:")
print(dt_grid.best_params_)

print("\nBest CV Accuracy:")
print(dt_grid.best_score_)

% Code Cell 83
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score

```

```

)

y_pred_best_dt = best_dt.predict(X_test)

best_dt_accuracy = accuracy_score(
    y_test,
    y_pred_best_dt
)

best_dt_precision = precision_score(
    y_test,
    y_pred_best_dt
)

best_dt_recall = recall_score(
    y_test,
    y_pred_best_dt
)

best_dt_f1 = f1_score(
    y_test,
    y_pred_best_dt
)

print("Optimized Decision Tree")
print("-----")
print("Accuracy :", best_dt_accuracy)
print("Precision:", best_dt_precision)
print("Recall   :", best_dt_recall)
print("F1 Score :", best_dt_f1)

% Code Cell 84
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import accuracy_score, f1_score

# Create 5 folds
cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

fold_results = []

for fold, (train_index, val_index) in enumerate(
    cv.split(X_train, y_train), start=1
):

```

```

# Split training data into fold-training and fold-validation
X_fold_train = X_train.iloc[train_index]
X_fold_val = X_train.iloc[val_index]

y_fold_train = y_train.iloc[train_index]
y_fold_val = y_train.iloc[val_index]

# Decision Tree
dt_fold = best_dt
dt_fold.fit(X_fold_train, y_fold_train)

dt_pred = dt_fold.predict(X_fold_val)

dt_acc = accuracy_score(
    y_fold_val,
    dt_pred
)

dt_f1 = f1_score(
    y_fold_val,
    dt_pred
)

# Random Forest
rf_fold = best_rf
rf_fold.fit(X_fold_train, y_fold_train)

rf_pred = rf_fold.predict(X_fold_val)

rf_acc = accuracy_score(
    y_fold_val,
    rf_pred
)

rf_f1 = f1_score(
    y_fold_val,
    rf_pred
)

fold_results.append([
    fold,
    dt_acc,
    dt_f1,
    rf_acc,
    rf_f1
])

```

```

fold_comparison = pd.DataFrame(
    fold_results,
    columns=[
        "Fold",
        "Decision Tree Accuracy",
        "Decision Tree F1",
        "Random Forest Accuracy",
        "Random Forest F1"
    ]
)

fold_comparison

% Code Cell 85
comparison = pd.DataFrame({
    "Model": [
        "Decision Tree",
        "Random Forest"
    ],

    "Accuracy": [
        best_dt_accuracy,
        best_rf_accuracy
    ],

    "Precision": [
        best_dt_precision,
        best_rf_precision
    ],

    "Recall": [
        best_dt_recall,
        best_rf_recall
    ],

    "F1 Score": [
        best_dt_f1,
        best_rf_f1
    ]
})

comparison

% Code Cell 86
dt_prob = best_dt.predict_proba(X_test)[: , 1]

```



```

rf_prob = best_rf.predict_proba(X_test)[: , 1]

% Code Cell 87
dt_fpr, dt_tpr, _ = roc_curve(
    y_test,
    dt_prob
)

rf_fpr, rf_tpr, _ = roc_curve(
    y_test,
    rf_prob
)

% Code Cell 88
dt_auc = roc_auc_score(
    y_test,
    dt_prob
)

rf_auc = roc_auc_score(
    y_test,
    rf_prob
)

print("Decision Tree AUC :", dt_auc)
print("Random Forest AUC :", rf_auc)

% Code Cell 89
dt_auc = roc_auc_score(
    y_test,
    dt_prob
)

rf_auc = roc_auc_score(
    y_test,
    rf_prob
)

print("Decision Tree AUC :", dt_auc)
print("Random Forest AUC :", rf_auc)

% Code Cell 90
final_results = pd.DataFrame({
    "Model": [
        "Decision Tree",
        "Random Forest"
    ],

```

```
"Accuracy": [
    best_dt_accuracy,
    best_rf_accuracy
],

"Precision": [
    best_dt_precision,
    best_rf_precision
],

"Recall": [
    best_dt_recall,
    best_rf_recall
],

"F1 Score": [
    best_dt_f1,
    best_rf_f1
],

"AUC": [
    dt_auc,
    rf_auc
]
})

final_results

% Code Cell 91
```